Implementación del Algoritmo CKY

Práctica de Programación y Algoritmia Avanzada – GIA

Integrantes: Ilyas Ouacham Rhailane
Laia Beyloc Guzmán

Grupo: 12

15 de junio de 2026

Índice

1. Enumeración y Arquitectura de Ficheros **2**

2. Explicación de los Códigos Desarrollados **2**
2.1. Pipeline principal 2
2.2. Lectura de datos 3
2.3. CKY 3

3. Formato Elegido para Describir Gramáticas **4**

4. Descripción de las Extensiones Realizadas **5**
4.1. Conversor a Forma Normal de Chomsky 5
4.2. CKY probabilístico (PCKY) 5

5. Valoración del Aprendizaje Obtenido **6**

6. Otras Consideraciones Pertinentes **6**

Referencias **6**

1. Enumeración y Arquitectura de Ficheros

A continuación se describe la estructura modular del proyecto:

- `main.py`: archivo principal que gestiona la interfaz y la entrada, y utiliza el resto de archivos para producir el resultado.
- `cky.py`: módulo que implementa el algoritmo determinista CKY.
- `pcky.py`: módulo de extensión que contiene la variante probabilística del algoritmo CKY.
- `utils/`
  - `cli.py`: define y configura la interfaz de línea de comandos y la gestión de argumentos.
  - `grammar_parser.py`: lee, limpia y transforma el archivo `.txt` en un diccionario que representa la gramática.
  - `cnf_converter.py`: módulo de extensión que detecta y transforma cualquier Gramática Libre de Contexto (CFG) a Forma Normal de Chomsky (CNF).
- `datos/`: directorio que almacena las gramáticas de prueba y los archivos de palabras para la ejecución en lote.
- `tests/`: directorio que contiene los tests en `test_cky.py` para validar el código mediante la librería `pytest`.

2. Explicación de los Códigos Desarrollados

2.1. Pipeline principal

El flujo de trabajo se encuentra en el archivo `main.py`, que hace uso de otros archivos manteniendo la estructura modular. La ejecución se inicia mediante el siguiente comando en la línea de comandos:

```
main.py [--pcky] [--cnf] [--batch <archivo>] [--frase] [--
   help]  gramatica [palabra]
```

El programa sigue el siguiente orden secuencial:

- **Validación de argumentos:** comprueba que haya suficientes datos para trabajar.
- **Lectura de la gramática:** lee el archivo de gramática indicado y devuelve un diccionario para representarla.
- **Comprobación de CNF y potencial transformación:** comprueba si la gramática está en CNF y, en caso necesario, la transforma.
- **Validación de coherencia:** comprueba que los argumentos sean coherentes con la naturaleza de la gramática.
- **Obtención de la entrada:** lee la palabra dada, o el archivo con palabras si se indica con el flag `-batch`.
- **Ejecución del algoritmo:** una vez leídos los datos, ejecuta el algoritmo correspondiente entre CKY y PCKY.

- **Muestra de resultados:** imprime los resultados de manera estructurada en la terminal.

2.2. Lectura de datos

La lectura de la gramática se realiza con la función `leer_gramática` de `grammar_parser.py`. Esta función ignora líneas vacías y espacios de más, así como comentarios denotados con el marcador "#" (como en Python). Obtiene secuencialmente la gramática fila a fila: en cada regla comprueba si hay alguna probabilidad (implementado para el PCKY, aunque el lector de gramática es general) y guarda la regla en un diccionario donde la llave es el símbolo no terminal generador y el valor es una tupla con la producción (cada elemento es un símbolo) y la probabilidad de la regla.

Finalmente devuelve la gramática y una variable booleana que indica si la gramática es probabilística, para la comprobación de coherencia posterior.

Un ejemplo de representación con diccionario para la gramática G:

```
G =
  S -> AB
  A -> a
  B -> b

gramatica = {
  'S': [(('A', 'B'), 1.0)],
  'A': [(('a',), 1.0)],
  'B': [(('b',), 1.0)]
}
```

2.3. CKY

El módulo principal implementa el análisis sintáctico mediante programación dinámica *bottom-up*. El flujo de ejecución se divide en cuatro fases:

- **Preprocesamiento de la gramática:** para optimizar la búsqueda se invierte el diccionario. Al ser la tabla construida *bottom-up*, se busca encontrar los no terminales que generan la producción actual; por ello el formato original resulta subóptimo y se recurre a la inversión para convertir la búsqueda en un acceso directo $O(1)$. Adicionalmente, se eliminan las probabilidades, que son exclusivas del algoritmo PCKY.
- **Inicialización de la tabla:** se define una matriz de dimensiones $(N+1) \times N$, donde N es la longitud de la palabra. Las filas representan la longitud de la subpalabra (L) y las columnas el índice de inicio (i). Se inicializa la base ($L = 1$) evaluando cada carácter individualmente para extraer los no terminales que lo derivan (reglas del tipo $A \to a$).
- **Iteración principal:** se rellena el resto de la matriz nivel a nivel mediante tres bucles anidados. Una ventana de longitud L se desliza sobre la palabra (comienza

en el índice i, que aumenta de uno en uno). Para cada subpalabra se evalúan todas sus particiones binarias (el índice k representa dónde comienza la partición, desde $k = 1$ hasta $k = L - 1$). El contenido de la celda actual se calcula realizando el producto cartesiano de los no terminales encontrados en las celdas particionadas y verificando si cada combinación resultante existe en la gramática.

- **Resultado:** al terminar el llenado, la celda superior corresponde a $L = N$. Si el símbolo inicial de la gramática está contenido en esta celda, la función devuelve `True`; en caso contrario, `False`. Esta lógica refleja que la presencia del símbolo inicial al final de la ejecución implica que se ha podido "escalar" desde la palabra hasta dicho símbolo mediante reglas de la gramática (el proceso inverso de la generación), lo que demuestra que la palabra pertenece al lenguaje.

Para ilustrar la iteración principal, consideremos la palabra `aaabbb`: al evaluar la longitud $L = 4$ en la posición $i = 1$, se analiza la subpalabra `aabb`. El algoritmo itera sobre sus tres particiones posibles ($k = 1, 2, 3$): `a|abb`, `aa|bb` y `aab|b`. Se consultan las celdas previamente calculadas para esas particiones, se evalúa el producto cartesiano de todas las parejas y se rellena la celda actual con los no terminales que pueden generar alguna de las combinaciones.

3. Formato Elegido para Describir Gramáticas

El formato elegido para describir las gramáticas es un derivado de la **Forma de Backus-Naur**, diseñado para ser más fácil de leer. Las reglas sintácticas se rigen por las siguientes convenciones (asumiendo siempre la separación por, al menos, un espacio en blanco):

- **Operador de derivación:** la regla se divide en dos partes separadas por una flecha, denotada como `->`.
- La parte izquierda contiene exclusivamente un símbolo no terminal (el generador).
- La parte derecha contiene una o múltiples producciones separadas por el operador `|`.
- Los símbolos dentro de una producción se delimitan entre sí mediante espacios. Esto permite trabajar tanto con símbolos de un carácter (p. ej., `a`) como con palabras (p. ej., `pelicano`). En el caso de las gramáticas CNF, una producción solo puede componerse de dos no terminales o de un terminal.
- El símbolo vacío se representa con una `e` minúscula.
- Se puede definir una PCFG añadiendo una probabilidad a cada producción, escrita al final entre corchetes `[<prob>]` (p. ej., `A B [0.8]`).

Ejemplo de gramática probabilística:

```
S -> A B [0.5]
A -> a [0.7] | b [0.3]
B -> b [1.0]
```

4. Descripción de las Extensiones Realizadas

4.1. Conversor a Forma Normal de Chomsky

Este módulo contiene dos funciones principales: `es_cnf`, encargada de detectar si una gramática está en CNF, y `convertir_a_cnf`, que realiza la transformación (con una función auxiliar por cada paso).

La primera función es sencilla: comprueba que todas las reglas sean del tipo $A \to BC$ o $A \to a$; si alguna incumple esta condición, la gramática no está en CNF. Nótese que esta llamada puede sustituirse por el flag `-cnf` (se indica la línea a cambiar en el código).

La segunda sigue los pasos siguientes para transformar cualquier CFG a CNF:

- **Crear nuevo símbolo inicial** (habitualmente redundante): se crea un símbolo nuevo (p. ej., `S0`) y una regla $S0 \to S$. Resulta necesario en casos donde el símbolo inicial aparece en alguna producción.
- **Aislar terminales:** en una regla como $S \to aB$, se crea un no terminal X_1, se modifica la regla a $S \to X_1 B$ y se añade $X_1 \to a$.
- **Binarizar reglas n-arias:** dada la regla $S \to ABC$, se modifica a $S \to AD$ y se añade $D \to BC$.
- **Eliminar caracteres vacíos:** se eliminan todos los símbolos vacíos y las reglas vacías (p. ej., $A \to e$).
- **Eliminar reglas unitarias:** aplicando la propiedad transitiva, si $A \to B$ y $B \to c$, la regla se convierte en $A \to c$.
- **Eliminar reglas inalcanzables:** paso añadido a posteriori, tras comprobar que algunas reglas quedaban inalcanzables tras los pasos anteriores.

La mayoría de estos pasos fueron extraídos del capítulo 18 de *Speech and Language Processing* (Jurafsky & Martin, 2023). Las excepciones son: creación de nuevo símbolo inicial, eliminación de caracteres vacíos y eliminación de reglas inalcanzables, añadidas porque el libro no contempla el símbolo vacío ni el caso en que el símbolo inicial aparece en una producción.

4.2. CKY probabilístico (PCKY)

Este módulo contiene toda la lógica del algoritmo CKY probabilístico. Comparte la estructura básica con CKY, con las siguientes diferencias:

- Al invertir la gramática no se elimina la probabilidad.
- En cada celda se almacena, además de los símbolos, la probabilidad y un *back-pointer*.
- En cada paso, al añadir un símbolo, se multiplican las probabilidades de las dos celdas anteriores y la probabilidad de emisión del símbolo; en caso de ambigüedad, se escoge la regla con mayor probabilidad.
- Al finalizar, en lugar de devolver un booleano, se devuelve la probabilidad de generar la palabra.

5. Valoración del Aprendizaje Obtenido

A lo largo de esta práctica hemos aprendido enormemente. No solo hemos mejorado nuestras habilidades en programación dinámica y programación en general, sino que hemos profundizado en el análisis sintáctico, lo que ha complementado muy bien nuestra asignatura de Procesamiento del Lenguaje Humano, donde parte del temario fue dedicado al análisis sintáctico con CKY y PCKY. De hecho, algunas de las adiciones (como el árbol sintáctico) fueron inspiradas por la combinación de ambas asignaturas.

Además, hemos aprendido cosas nuevas menos relacionadas con la práctica, como el diseño de interfaces de línea de comandos con argumentos y flags; así como la realización de tests automatizados.

En conclusión, ha sido una práctica muy acertada, cuyo impacto en este cuatrimestre ha sido sustancial, ya que ha ayudado a reforzar y consolidar una parte de uno de los focos actuales de la carrera: el procesamiento del lenguaje natural (NLP).

6. Otras Consideraciones Pertinentes

Además de los requisitos del enunciado, hemos implementado las siguientes funcionalidades:

- **Procesamiento en lote:** mediante el flag `-batch` se puede especificar un archivo del que extraer palabras para ejecutar el algoritmo con múltiples entradas en una sola ejecución.
- **Analizador sintáctico:** en el algoritmo PCKY no nos hemos limitado a calcular probabilidades, sino que hemos creado árboles sintácticos mediante *backpointers*. Asimismo, hemos preparado una gramática (`gp4.txt`) con reglas sintácticas del castellano para probar el análisis de frases. Para procesar frases es necesario el flag `-frase` (o `-f`), así como envolver la frase entre comillas (p. ej., `python main.py -p -f datos/gp4.txt "kenny duerme en el sofa"`).
- **Módulo de testing automatizado:** implementado con `pytest`, con un total de 123 tests (combinando tests con múltiples ejecuciones por test). Este conjunto es suficiente para demostrar que el algoritmo funciona sin fallos ni errores de ejecución.

Referencias

Jurafsky, D., & Martin, J. H. (2023). *Speech and Language Processing* (3.ª ed.). Prentice Hall. https://web.stanford.edu/~jurafsky/slp3/

Wikipedia contributors. (2024). CYK algorithm. https://en.wikipedia.org/wiki/CYK_algorithm